Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform

Coding Standards v2

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Introduction

This document outlines the coding standards, conventions and a dev team starting guide for the Hackathon Platform. These standards will ensure uniformity, clarity and efficiency across our system.

Tech stack:

- **Frontend:** Angular with Cypress E2E
- **Backend:** Java Spring Boot
- **Database:** PostgreSQL
- **CI/CD:** Github Actions

Repository Structure

```
├── .github/workflows/ci.yml                    GitHub Actions CI pipeline
├── backend/                                    Spring Boot Java API
│   ├── pom.xml                                 Maven build file + all dependencies
│   ├── checkstyle.xml                          Checkstyle rules for Java code
│   └── src/main/java/com/hackathon/platform/HackathonPlatformApplication.java   app entry point
│       │   └── resources/
│       │       ├── application.properties      main config
│       │       ├── application-test.properties test config
│       │       └── db/migration/               Flyway migration
│       └── test/java/com/hackathon/platform/HackathonPlatformApplicationTests.java
├── frontend/
│   ├── .eslintrc.json                          ESLint rules for TypeScript
│   ├── nginx.conf                              Nginx config for the Docker container
│   └── src/app/
├── docker/                                     builds Spring Boot, Angular Docker image
├── docker-compose.yml                          local dev: PostgreSQL + Redis
├── lighthouserc.json                           Lighthouse CI thresholds
├── sonar-project.properties                    SonarCloud config
```

Local Development

The following dependencies need to be installed before starting development:

- **Java 21** - https://adoptium.net/temurin/releases/?version=21
- **Maven 3.9 or later:** https://maven.apache.org/download.cgi (add this to your PATH environment variable)
- **Node.js 20 LTS:** https://nodejs.org/en/download
- **Docker Desktop:** https://www.docker.com/products/docker-desktop

Workflow

Terminal 1: docker compose up -d. This starts postgres and redis.

Terminal 2: cd backend && maven spring-boot:run. This starts the spring boot backend on port 8080.

Terminal 3: cd frontend && npm start. This starts the Angular project on port 4200.

Ensure that there aren't any other services using the same ports, else these commands will not work.

Docker

File: docker-compose.yml

This runs two services, the PostgreSQL and Redis service. docker compose up -d runs both containers.

File: docker/backend.Dockerfile

This file is used by the CI to define how to build a Docker image of the spring boot app. It uses a multi-staged build, the second stage copies only the output of the first. For efficiency the dependencies are cached and are only downloaded if the pom.xml changes. For security, the final image only has the JRE and JAR, no maven or source code.

File: docker/frontend.Dockerfile

This file has the same purpose as the backend Dockerfile. The build produces a folder of static HTML, CSS and JS files. Nginx, which is a web server, serves those files, Node is not needed at runtime.

File: frontend/nginx.conf

Angular is a Single Page Application. All routing happens in the browser via JS, the server just serves the index.html and Angular routes the rest. This file tells Nginx to serve the exact file, if it is not found serve index.html.

Spring boot backend setup

File: backend/pom.xml (Project Object Model)

This defines the project entity, all its dependencies and build configuration for Maven.

Dependencies

Dependancy	Purpose
spring-boot-starter-web	Contains everything needed for the REST API, it also has an embedded Tomcat server, Spring MVC for @RestController and Jackson for JSON serialisation
spring-boot-starter-security	Spring security framework, used for authentication, authorization which sits under JWTs
jjwt-api/impl/jackson	JSON web token library
spring-boot-starter-data-jpa	Spring Data JPA & Hibernate, allows definition of @Entity classes and @Repository interfaces. (DO NOT WRITE SQL MANUALLY)
PostgreSQL	The JDBC Driver connects the app to the DB over the network.
Flyway	Database migration tool. Runs versioned SQL files on startup which helps keep schema in version control.
spring-boot-starter-data-redis	Redis client for Spring.
spring-boot-starter-validation	Bean validation, allows @NotNull, @Email, @Size on request DTOs.
spring-boot-starter-actuator	Starts /actuator/health, this is used by Docker healthchecks and Azure load balancer probes.

lombok	Annotation processor that generates boilerplate code. Allows @Getter, @Builder, @RequiredArgsConstructor.

Test Dependencies

Dependency	Purpose
spring-boot-starter-test	JUnit 5, Mockito, MockMvc, AssertJ. These are used for unit tests and integration tests.
spring-security-test	Allows utilities for testing secured endpoints.

Build Plugins

Plugin	Purpose
spring-boot-maven-plugin	Packages the app as a JAR. Allows maven spring-boot:run command.
jacoco-maven-plugin	Generates a code coverage report.
spotless-maven-plugin	Auto formats Java source code to Google Java style. Use maven spotless:apply to fix your code, use maven spotless:check to check your code. (Linting)
maven-checkstyle-plugin	Enforces structural coding rules, checkstyle.xml is the config. (linting)

File: HackathonPlatformApplication.java

This is the app entry point into the system. The class is annotated with @SpringBootApplication. This is a shorthand for the following:

- @Configuration: Defines the Spring beans
- @EnableAutoConfiguration: Auto configuration of beans based on the classpath.
- @ComponentScan: Scans the sub-packages for @Component, @Service etc.

File: application.properties

This is the configuration that spring boot reads on startup. This is the file that contains all the settings for spring boot.

File: application-test.properties

This is similar to the file above, however it contains a few adjusted settings that are needed to make testing possible and efficient. When a class with the annotation @ActiveProfiles(“test”) is detected, this file overrides the normal config file. The following changes are the most significant:

- Redis auto config excluded: Prevents tests failing due to Redis server issues.

Database Set up

There are three different environments for the DB:

Environment	Database	Schema management
Local dev	Docker PostgreSQL (localhost:5432)	Flyway, on backend startup
CI	Github Actions postgres service container	
Azure	Azure Database for PostgreSQL Flexible Server	Flyway on deployment

Flyway

Flyway is a DB migration tool, we will not manually run SQL in pgAdmin, .sql files in the repo are automatically run when the backend starts. This also helps us have versioned .sql files. These files are at: backend/src/main/resources/db/migration/ .

Naming: V{number}__{short description}.sql(without brackets, 2 underscores)

Flyway has a table called flyway_schema_history in the DB. Everytime the backend starts it checks which migrations are already applied and runs new ones in order. NEVER edit a migration file that has already been applied. If you need to change something, write a new migration file. If you edit an applied migration it causes a checksum mismatch and will break the project.

If you need to make a schema change follow these steps:

1. Create a .sql file with your SQL statements.
2. Commit and push.
3. Make the group aware of your change on Discord.
4. When anyone runs the backend locally it's then applied.
5. When we deploy to Azure, Flyway will apply it.

CI/CD

CI Pipeline

This runs when someone pushes to main or dev, or when there is a PR into main or dev.

Job 1: Backend

This creates a real PostgreSQL container as a service, this is not the same container that is started manually. The backend tests connect to this real DB.

1. Checkout code - Downloads the repo onto the runner.
2. Set up Java 21
3. Spotless check - fails any Java files that are not formatted to Google styles. If this fails, fix your code with: maven spotless:apply
4. Checkstyle - fails if any Java file violates the rules in the .xml config
5. Build and test - mvn clean verify compiles everything, runs all JUnit tests and generates a Jacoco coverage report.
6. Upload coverage - saves the Jacoco XML as a workflow artifact so SoncarCloud can download it and read it.

Job 2: Frontend

1. Checkout code
2. Set up Node 20
3. npm ci - installs all Angular dependencies
4. ESLint - npm run lint, will fail if any Typescript or HTML violates the rules.
5. Unit tests - npm run test – –watch=false – browsers=ChromeHeadless runs all Karma/Jasmine in a headless Chrome browser.
6. Production build - uses npm run build – –configuration=production, this compiles Angular
7. Upload build artifact - saves the compiled dist/ folder for Lighthouse

Job 3: Lighthouse

Runs after the frontend completes. This builds the Angular project for production and runs the Lighthouse CI against the compiled static files. Thresholds are configured in lighthouse.json.

Job 4: Playwright E2E tests

After the frontend and backend builds have successfully completed, E2E tests are run on Chromium using the built packages.

Job 5: k6 load tests

Using Grafana k6, a load test is automatically run.

Job 6: CodeCov

This job runs all backend tests, including integration and unit tests, and frontend tests. It then uploads the results to CodeCov where they can be viewed in the CodeCov dashboard.

CI Pipeline

CD automatically builds the frontend and backend and deploys it to the Azure Container App. The images are first built using Docker Buildx, they are given unique tags using the current timestamp and a short SHA hash. The images are then pushed to the Azure Container Registry, thereafter if it is successful the images are deployed to Azure Container Apps. A health check then checks for the 'UP' status and completes successfully once it is found.

Branch Protection

- Requires PR before merging. The main branch PR requires 3 reviews before merging, dev branch needs 2 reviews.

- Status checks need to pass before merging

Code Quality Checks

Spotless - Java formatter

Automatically formats Java code to Google Java style. This includes indentation, bracket placements, line wrapping, imports.

How to use it:

mvn spotless:apply Fixes your code, run this before you PR

mvn spotless:check checks without fixing (CI runs this one)

Java rules

This enforces structural coding rules, such as naming styles, import rules, complexity.

Rule	Checks for
TypeName	Classes must be PascalCase
MethodName/variableName	Methods must be camelCase
ConstantName	Constants must be UPPER_SNAKE_CASE
AvoidStarImports	No imports with star, must import explicitly
UnusedImports	Do not keep unused imports, they are a security risk and create bloatware
NeedBraces	If, for, while always need {}. This helps increase readability and consistency.
LineLength	Max 120 chars per line
CyclomaticComplexity	Max 15, methods that are too complex should be split up
StringLiteralEquality	Use .equals(), not == for string. This

	increases reliability, especially in Java.
EqualsHashCode	If you override equals(), hashCode() also should be overridden.
MissingJavaDocMethod	Public methods must have a Javadoc comment (this can be short).
Packages	Lowercase dotted, eg. com.hackathon.platform.auth
Enums	PascalCase, UPPER_SNAKE, eg. Status { PENDING, APPROVED }

Command: mvn checkstyle:check

ESLint - Typescript/Angular Linting

Element	Rule	Example
Components	PascalCase + Component	LoginComponent
Services	PascalCase + Service	AuthService
Interfaces	PascalCase	UserProfile
Enums	PascalCase	UserRole { Admin, User}
Files	kebab-case with type	login.component.ts
Variables/Methods	camelCase	currentUser
Constants	UPPER_SNAKE_CASE	API_BASE_URL
CSS Classes	kabab-case	event-card

Commands:

npm run lint	Checks linting without fixing
ng lint –fix	Fixes fixable linting errors

Testing

Unit tests (JUnit/ Karma +Jasmine)

Backend: backend/src/test/java/

Frontend: frontend/src/**/*.spec.ts

Unit tests are for single class or function isolation. If you need an external dependency such as the DB, use mocks with Mockito (backend) or Jasmine spies (frontend).

How to run:

Backend: mvn test

Frontend: npm test

The context loads test HackathonPlatformApplicationTests.java, this is a special case. It's a smoke test, there's no logic, it just starts the entire app and checks that all beans wire together. Ensure @Autowired is correct.

Integration Tests (Spring Boot Test)

[Will be expanded later on]

Tests should use @SpringBootTests, this will start a spring context and uses a real DB.

End to end tests (Playwright)

Config: playwright.config.ts

Tests: e2e/

Playwright launches a browser and interacts with the application.

Run locally:

npm run test:e2e

Jacoco

The report is generated at backend/target/site/jacoco/. The index.html shows the coverage by class and method.

Lighthouse

This is used for performance and accessibility testing. The report is stored in the workflow artifacts.

Package Structure

This is organised by layer, eg. all controllers, all services etc are in their own folders.

Additional Information - Java

Controllers only handle HTTP, do not include any logic here.

Example:

```java
//delegates to service, returns ResponseEntity with status
@PostMapping("/events")
public ResponseEntity<EventResponse> createEvent(@Valid @RequestBody CreateEventRequest request) {
    EventResponse response = eventService.createEvent(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```


Use @Valid on @RequestBody to trigger Bean Validation.

Do not use any magic numbers. All numbers need to be assigned to a private static final var.

NEVER throw null in from service methods because then the caller needs to have a null check. Instead throw an exception.

Additional Information - Angular

HTTP calls only go into service files.

Example:

Always clean up subscriptions. If they are not cleaned it could possibly cause memory leaks because the subscription keeps running after the component is destroyed.

Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform

Coding Standards v1

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Introduction

This document outlines the coding standards, conventions and a dev team starting guide for the Hackathon Platform. These standards will ensure uniformity, clarity and efficiency across our system.

Tech stack:

- **Frontend:** Angular with Cypress E2E
- **Backend:** Java Spring Boot
- **Database:** PostgreSQL
- **CI/CD:** Github Actions

Repository Structure

```
├── .github/workflows/ci.yml                GitHub Actions CI pipeline
├── backend/                                Spring Boot Java API
│   ├── pom.xml                             Maven build file + all dependencies
│   ├── checkstyle.xml                      Checkstyle rules for Java code
│   └── src/main/java/com/hackathon/platform/HackathonPlatformApplication.java   app entry point
│       │   └── resources/
│       │       ├── application.properties          main config
│       │       ├── application-test.properties     test config
│       │       └── db/migration/                   Flyway migration
│       └── test/java/com/hackathon/platform/HackathonPlatformApplicationTests.java
├── frontend/
│   ├── .eslintrc.json                      ESLint rules for TypeScript
│   ├── nginx.conf                          Nginx config for the Docker container
│   └── src/app/
├── docker/                                 builds Spring Boot, Angular Docker image
├── docker-compose.yml                      local dev: PostgreSQL + Redis
├── lighthouserc.json                       Lighthouse CI thresholds
├── sonar-project.properties                SonarCloud config
```

Local Development

The following dependencies need to be installed before starting development:

- **Java 21** - https://adoptium.net/temurin/releases/?version=21
- **Maven 3.9 or later:** https://maven.apache.org/download.cgi (add this to your PATH environment variable)
- **Node.js 20 LTS:** https://nodejs.org/en/download
- **Docker Desktop:** https://www.docker.com/products/docker-desktop

Workflow

Terminal 1: docker compose up -d. This starts postgres and redis.

Terminal 2: cd backend && maven spring-boot:run. This starts the spring boot backend on port 8080.

Terminal 3: cd frontend && npm start. This starts the Angular project on port 4200.

Ensure that there aren't any other services using the same ports, else these commands will not work.

Docker

File: docker-compose.yml

This runs two services, the PostgreSQL and Redis service. docker compose up -d runs both containers.

File: docker/backend.Dockerfile

This file is used by the CI to define how to build a Docker image of the spring boot app. It uses a multi-staged build, the second stage copies only the output of the first. For efficiency the dependencies are cached and are only downloaded if the pom.xml changes. For security, the final image only has the JRE and JAR, no maven or source code.

File: docker/frontend.Dockerfile

This file has the same purpose as the backend Dockerfile. The build produces a folder of static HTML, CSS and JS files. Nginx, which is a web server, serves those files, Node is not needed at runtime.

File: frontend/nginx.conf

Angular is a Single Page Application. All routing happens in the browser via JS, the server just serves the index.html and Angular routes the rest. This file tells Nginx to serve the exact file, if it is not found serve index.html.

Spring boot backend setup

File: backend/pom.xml (Project Object Model)

This defines the project entity, all its dependencies and build configuration for Maven.

Dependencies

Dependancy	Purpose
spring-boot-starter-web	Contains everything needed for the REST API, it also has an embedded Tomcat server, Spring MVC for @RestController and Jackson for JSON serialisation
spring-boot-starter-security	Spring security framework, used for authentication, authorization which sits under JWTs
jjwt-api/impl/jackson	JSON web token library
spring-boot-starter-data-jpa	Spring Data JPA & Hibernate, allows definition of @Entity classes and @Repository interfaces. (DO NOT WRITE SQL MANUALLY)
PostgreSQL	The JDBC Driver connects the app to the DB over the network.
Flyway	Database migration tool. Runs versioned SQL files on startup which helps keep schema in version control.
spring-boot-starter-data-redis	Redis client for Spring.
spring-boot-starter-validation	Bean validation, allows @NotNull, @Email, @Size on request DTOs.
spring-boot-starter-actuator	Starts /actuator/health, this is used by Docker healthchecks and Azure load balancer probes.

lombok	Annotation processor that generates boilerplate code. Allows @Getter, @Builder, @RequiredArgsConstructor.

Test Dependencies

Dependency	Purpose
spring-boot-starter-test	JUnit 5, Mockito, MockMvc, AssertJ. These are used for unit tests and integration tests.
spring-security-test	Allows utilities for testing secured endpoints.
H2	In memory DB used during unit tests in CI so that a real PostgreSQL connection is not needed.

Build Plugins

Plugin	Purpose
spring-boot-maven-plugin	Packages the app as a JAR. Allows maven spring-boot:run command.
jacoco-maven-plugin	Generates a code coverage report.
spotless-maven-plugin	Auto formats Java source code to Google Java style. Use maven spotless:apply to fix your code, use maven spotless:check to check your code. (Linting)
maven-checkstyle-plugin	Enforces structural coding rules, checkstyle.xml is the config. (linting)

File: HackathonPlatformApplication.java

This is the app entry point into the system. The class is annotated with @SpringBootApplication. This is a shorthand for the following:

- @Configuration: Defines the Spring beans
- @EnableAutoConfiguration: Auto configuration of beans based on the classpath.
- @ComponentScan: Scans the sub-packages for @Component, @Service etc.

File: application.properties

This is the configuration that spring boot reads on startup. This is the file that contains all the settings for spring boot.

File: application-test.properties

This is similar to the file above, however it contains a few adjusted settings that are needed to make testing possible and efficient. When a class with the annotation @ActiveProfiles(“test”) is detected, this file overrides the normal config file. The following changes are the most significant:

- DB URL changed to the H2 in memory DB, no real DB is used for unit tests.
- Redis auto config excluded: Prevents tests failing due to Redis server issues.

Database Set up

There are three different environments for the DB:

Environment	Database	Schema management
Local dev	Docker PostgreSQL (localhost:5432)	Flyway, on backend startup
CI	Github Actions postgres service container	
Azure	Azure Database for PostgreSQL Flexible Server	Flyway on deployment

Flyway

Flyway is a DB migration tool, we will not manually run SQL in pgAdmin, .sql files in the repo are automatically run when the backend starts. This also helps us have versioned .sql files. These files are at: backend/src/main/resources/db/migration/ .

Naming: V{number}__{short description}.sql(without brackets, 2 underscores)

Flyway has a table called flyway_schema_history in the DB. Everytime the backend starts it checks which migrations are already applied and runs new ones in order. NEVER edit a migration file that has already been applied. If you need to change something, write a new migration file. If you edit an applied migration it causes a checksum mismatch and will break the project.

If you need to make a schema change follow these steps:

6. Create a .sql file with your SQL statements.
7. Commit and push.
8. Make the group aware of your change on Discord.
9. When anyone runs the backend locally it's then applied.
10. When we deploy to Azure, Flyway will apply it.

CI/CD

[CD is not implemented yet]

CI Pipeline

This runs when someone pushes to main or dev, or when there is a PR into main or dev.

Job 1: Backend

This creates a real PostgreSQL container as a service, this is not the same container that is started manually. The backend tests connect to this real DB.

7. Checkout code - Downloads the repo onto the runner.
8. Set up Java 21
9. Spotless check - fails any Java files that are not formatted to Google styles. If this fails, fix your code with: maven spotless:apply
10. Checkstyle - fails if any Java file violates the rules in the .xml config
11. Build and test - mvn clean verify compiles everything, runs all JUnit tests and generates a Jacoco coverage report.
12. Upload coverage - saves the Jacoco XML as a workflow artifact so SoncarCloud can download it and read it.

Job 2: Frontend

8. Checkout code
9. Set up Node 20
10. npm ci - installs all Angular dependencies
11. ESLint - npm run lint, will fail if any Typescript or HTML violates the rules.
12. Unit tests - npm run test – –watch=false – browsers=ChromeHeadless runs all Karma/Jasmine in a headless Chrome browser.
13. Production build - uses npm run build – –configuration=production, this compiles Angular
14. Upload build artifact - saves the compiled dist/ folder for Lighthouse

Job 3: SonarCloud [Disabled for now]

Runs after both backend and frontend complete (they run in parallel). Downloads the Jacoco report, then runs SonarCloud scanner against the codebase. It will analyse code quality, detect bugs, security issues and measure test coverage.

Job 4: Lighthouse

Runs after frontend completes. This builds the Angular project for production and runs the Lighthouse CI against the compiled static files. Thresholds are configured in lighthouse.json.

Branch Protection

- Requires PR before merging. The main branch PR requires 3 reviews before merging, dev branch needs 2 reviews.
- Status checks need to pass before merging.

Code Quality Checks

Spotless - Java formatter

Automatically formats Java code to Google Java style. This includes indentation, bracket placements, line wrapping, imports.

How to use it:

mvn spotless:apply Fixes your code, run this before you PR

mvn spotless:check checks without fixing (CI runs this one)

Java rules

This enforces structural coding rules, such as naming styles, import rules, complexity.

Rule	Checks for
TypeName	Classes must be PascalCase
MethodName/variableName	Methods must be camelCase
ConstantName	Constants must be UPPER_SNAKE_CASE
AvoidStarImports	No imports with star, must import explicitly
UnusedImports	Do not keep unused imports, they are a security risk and create bloatware
NeedBraces	If, for, while always need {}. This helps increase readability and consistency.
LineLength	Max 120 chars per line
CyclomaticComplexity	Max 15, methods that are too complex should be split up
StringLiteralEquality	Use .equals(), not == for string. This

	increases reliability, especially in Java.
EqualsHashCode	If you override equals(), hashCode() also should be overridden.
MissingJavaDocMethod	Public methods must have a Javadoc comment (this can be short).
Packages	Lowercase dotted, eg. com.hackathon.platform.auth
Enums	PascalCase, UPPER_SNAKE, eg. Status { PENDING, APPROVED }

Command: mvn checkstyle:check

ESLint - Typescript/Angular Linting

Element	Rule	Example
Components	PascalCase + Component	LoginComponent
Services	PascalCase + Service	AuthService
Interfaces	PascalCase	UserProfile
Enums	PascalCase	UserRole { Admin, User}
Files	kebab-case with type	login.component.ts
Variables/Methods	camelCase	currentUser
Constants	UPPER_SNAKE_CASE	API_BASE_URL
CSS Classes	kabab-case	event-card

Commands:

npm run lint	Checks linting without fixing
ng lint –fix	Fixes fixable linting errors

SonarCloud - Code Quality

[Placeholder, not used yet.]

Testing

Unit tests (JUnit/ Karma +Jasmine)

Backend: backend/src/test/java/

Frontend: frontend/src/**/*.spec.ts

Unit tests are for single class or function isolation. If you need an external dependency such as the DB, use mocks with Mockito (backend) or Jasmine spies (frontend).

How to run:

Backend: mvn test

Frontend: npm test

The context loads test HackathonPlatformApplicationTests.java, this is a special case. It's a smoke test, there's no logic, it just starts the entire app and checks that all beans wire together. Ensure @Autowired is correct.

Integration Tests (Spring Boot Test)

[Will be expanded later on]

Tests should use @SpringBootTests, this will start a spring context and uses a real DB.

End to end tests (Cypress)

Config: cypress.config.ts

Tests: cypress/e2e/smoke.cy,ts

Cypress launches a browser and interacts with the application.

Run locally:

npx cypress open Interactive mode

npx cypress run	Headless mode

Jacoco

The report is generated at backend/target/site/jacoco/. The index.html shows the coverage by class and method.

Lighthouse

This is used for performance and accessibility testing. The report is stored in the workflow artifacts.

Package Structure

This is organised by layer, eg. all controllers, all services etc are in their own folders.

Additional Information - Java

Controllers only handle HTTP, do not include any logic here.

Example:

```java
//delegates to service, returns ResponseEntity with status
@PostMapping("/events")
public ResponseEntity<EventResponse> createEvent(@Valid @RequestBody CreateEventRequest request) {
    EventResponse response = eventService.createEvent(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```


Use @Valid on @RequestBody to trigger Bean Validation.

Do not use any magic numbers. All numbers need to be assigned to a private static final var.

NEVER throw null in from service methods because then the caller needs to have a null check. Instead throw an exception.

Additional Information - Angular

HTTP calls only go into service files.

Example:

Always clean up subscriptions. If they are not cleaned it could possibly cause memory leaks because the subscription keeps running after the component is destroyed.